第一课-基础语法

本课程以 C++17 为语言上限。教学代码使用显式类型，标量采用等号初始化，普通对象采用圆括号构造；数组和聚合数据采用等号后的初始化列表。

这是一份可连续阅读的完整讲义。正文按去年仓库的演示顺序展开；每个实验包括概念推导、完整单文件程序、状态分析、错误定位及带答案的自检。配套题库另有 120 道独立例题，函数与指针练习随原第二课归位。

运行平台为 CPP-Classroom，选择 C++17，整份粘贴一个 .cpp 后运行。平台只运行程序，不内置本课程的自动判题。所有可变输入在题目左侧预填，或运行时按行发送；读到流结束的程序需要发送 EOF。

Original repository: unnc-aim/aim-2526-rookie-courses

章节目录

- 01. 从 Python 进入 C++ 程序

- 02. 命名空间与第一次输出

- 03. 输入、变量与输出格式

- 04. 运算符的逐步计算

- 05. 条件与 for 循环

- 06. 数组、std::array 与范围遍历

- 07. 字符串的查、改、增、删

- 08. 第一课整合与结构体预告

01 从 Python 进入 C++ 程序

原稿定位: README.md；Lesson 01/my_first_cplus.cpp：头文件与 main

1.1 源代码、编译和一次完整运行

去年课程从已经学过的 Python 出发：变量、条件、循环和函数不是全新的概念。现在改变的是表达规则和执行过程。Python 的缩进换成 C++ 的大括号，大部分语句末尾需要分号，变量的类型在编译时受到检查。学习目标是能读懂并修改战队工程中的 C++ 代码，而不是逐字把 Python 换一套拼写。

CPP-Classroom 中提交给编译器的是当前题目的一整份源文件。编译器先检查语法与类型，再生成可执行程序，平台随后运行该程序。点击运行不会从上次停止的位置继续，上一轮运行里的局部变量也不会留到下一轮。每次都从 main 重新开始。

本课程选择 C++17。完整示例均含所需的标准头文件和一个 main，可以整份粘贴进平台。不要把两份各有 main 的程序拼到同一题目中，也不要把终端里的 g++ 命令粘到 C++ 编辑区。后文的编译命令只用于讲师本地复核，学生课堂操作统一使用平台的运行按钮。

完整课堂实验

`L1_C01_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

执行推导

编译成功后 main 执行一次输出语句，写出一行文本，再返回 0。屏幕上看不到 return 的数值，因为它是进程退出状态，不是标准输出。

常见错误及原因

编译失败时先读第一条诊断及其行号。漏掉一个分号可能使后面的多行都被标红，不能据此判断每一行都错了。

自检

删掉输出语句末尾的分号，程序会先运行再报错吗？

参考解释： 不会。编译阶段未通过，平台不会执行这次有错误的源码。

1.2 头文件、main 与语句块

原始示例在文件顶部依次包含 iomanip、iostream、cmath、array 和 string。每个头文件提供某一组声明：iostream 声明输入输出对象，iomanip 提供输出格式控制，cmath 提供数学函数，array 和 string 提供相应的标准库类型。include 的作用不是“运行另一个程序”，而是让编译器看到需要的接口。

常规控制台程序从 main 进入。int 表示该函数向运行环境返回整数退出状态，圆括号是参数列表，大括号包围函数体。函数体中可以再出现大括号来建立局部作用域。离开作用域后，普通局部变量的生命周期结束。

原文件里的 #ifdef _WIN32 和 chcp 65001 是针对某些 Windows 终端的编码适配。CPP-Classroom 在独立 Linux 节点运行源程序，因此课堂版本不调用 system，也不暂停等待平台之外的 shell。中文解释保留在讲义里，自动核对的基础样例优先使用确定的 ASCII 输出。

完整课堂实验

`L1_C01_D02.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int value = 3;
    {
        int doubled = value * 2;
        std::cout << doubled << '\n';
    }
    std::cout << value << '\n';
    return 0;
}
```

执行推导

外层 value 一直存在，内层 doubled 只在内层块中有效，输出依次为 6 和 3。把最后一行改成输出 doubled 会产生编译错误。

常见错误及原因

不能因为某个平台间接包含了一个头文件，就把这种偶然依赖当成通用规则。完整示例明确列出自己使用的标准接口。

自检

为什么不能在同一个源文件里放两个 main?

参考解释： 这会重复定义程序入口，编译无法建立唯一的入口函数。

本章配套例题

ID	题目
S08	故障分层

02 命名空间与第一次输出

原稿定位: Lesson 01/stupid_namespace.cpp；new_new

2.1 std、using 与名字查找

原课程专门用 stupid_namespace.cpp 演示名字冲突。cout、cin、string 等标准库名字位于 std 命名空间。std::cout 说明“使用 std 中的 cout”，其中双冒号是作用域解析运算符。它和 Python 中模块前缀的直觉有相似之处，但不是同一套语言机制。

using namespace std 允许非限定地查找标准库名字，写法更短，也扩大了可能发生冲突的范围。原例又引入 mygo，其中有自己的 cout、cin，甚至一个名为 std 的结构体。此时简写 cout 不再清楚，std 也可能发生歧义。理解冲突比机械背诵“总是写 using”或“永远不许写 using”更重要。

原例用 ::std::cout 从全局作用域明确选择标准库。本课程完整示例通常直接写 std::，保留原课程的冲突实验供理解。在多文件工程中，尤其不要在公共头文件里放全局 using namespace，因为包含者也会受到影响。

完整课堂实验

L1_C02_D01.cpp · CPP-Classroom source

```
#include <iostream>

namespace aim {
int cout = 310;
}

int main() {
    std::cout << aim::cout << '\n';
    ::std::cout << "standard output\n";
    return 0;
}
```

执行推导

aim::cout 是我们定义的整数 310，std::cout 是输出流，两者只有拼写相同。第一行用输出流打印该整数，第二行打印文本。

常见错误及原因

不要把命名空间当成对象。std::cout 的 :: 负责选择名字，后续的 << 才把内容送入输出流。

自检

mygo 中存在名为 std 的类型，为什么 ::std::cout 仍能明确指向标准库？

参考解释： 开头的 :: 强制从全局作用域开始查找 std，避开当前引入的同名类型。

2.2 cout、换行与刷新

原始 new_new 只输出一句 Hello, C++!，这已经包含了后面所有输出的基本形状。左边是 std::cout，右边是被插入流的内容；多个 << 可以串起来，所以文本、整数和其他可输出对象能组成同一行。每个插入操作按表达式顺序发生。

反斜杠 n 表示换行。std::endl 除了换行还会刷新输出缓冲区，因此两者的语义并不完全相同。普通批量输出优先写换行；如果交互式程序在等待输入前需要确保提示立即发出，则刷新才与交互行为有关。不要把 endl 当作“每行结尾必须使用的关键字”。

字符常量用单引号，例如换行字符；字符串字面量用双引号，可以包含多个字符。输出的空格也属于结果的一部分。做样例对照时，核对每行的字段顺序、分隔空格和最后是否换行，而不是只看“数字似乎对了”。

完整课堂实验

L1_C02_D02.cpp · CPP-Classroom source

```
#include <iostream>

int main() {
    int id = 7;
    std::cout << "Hello, C++!\n";
    std::cout << "id=" << id << '\n';
    return 0;
}
```

执行推导

输出两行：Hello, C++! 与 id=7。字符串 id= 不会自动知道后面的变量，必须用另一个插入操作输出 id。

常见错误及原因

std::cout << "id" 会输出两个字母，不会读取名为 id 的变量。

自检

把第二行末尾的换行去掉，再输出 DONE，会出现几行?

参考解释： 第一句 Hello 单独一行，id=7 与 DONE 连在第二行，除非再显式插入分隔符或换行。

本章配套例题

ID	题目
S01	Hello 与退出码
S02	多个字段输出
S07	命名空间同名变量

03 输入、变量与输出格式

原稿定位: input_int；input_variable

3.1 整数输入与提取顺序

原 input_int 先创建 x、y，再执行 cin >> x >> y。C++ 的变量要先声明，编译器需要在读取前知道目标类型。int x = 0 将 x 初始化为 0；它与“读入一定成功”是两回事，初始化只是防止局部变量在未赋值时含不确定值。

流提取会跳过前导空白。输入 8 4、两行分别输入 8 与 4，或在两数间加入多个空格，都能得到同样的两个整数。输入字符串“8,4”则不同：逗号不是默认的整数分隔空白，第二次提取会失败。

CPP-Classroom 的预填输入会在文本发送完后关闭 stdin，适合确定数量的数据。选择运行时输入则可以按行发送，程序读够数据后自行结束；若程序循环读到输入结束，需要显式发送 EOF。EOF 表示输入流结束，不是把三个字母 E、O、F 当成数据输入。

完整课堂实验

L1_C03_D01.cpp · CPP-Classroom source

```
#include <iostream>

int main() {
    int x = 0, y = 0;
    if (!(std::cin >> x >> y)) {
        std::cout << "INVALID\n";
        return 1;
    }
    std::cout << x + y << ' ' << x - y << '\n';
    return 0;
}
```

标准输入：

```
8 4
```

执行推导

输入 8 4 时，先把 8 存进 x，再把 4 存进 y，输出 12 4。输入 8 robot 时，第二次提取失败，进入错误分支，不能继续把 y=0 当作合法测量值计算。

常见错误及原因

不要把编辑器中的换行与 stdin 中的换行混在一起。前者组织代码，后者只是本次运行的数据。

自检

输入 8、换行、4，与输入 8 空格 4 的结果相同吗？

参考解释： 对这两个整数的格式化提取相同；两种输入都提供两个合法 int。

3.2 double、char、string 与格式状态

原 input_variable 连续读取浮点数 a、单字符 b 和字符串 c。double 可以表示有小数的近似实数，char 存一个字符，std::string 管理字符序列。这是三个不同类型；变量名不决定类型，声明才决定。char 的 A 与字符串 A 在语法和类型上也不同。

fixed 与 setprecision(2) 放进输出流后，后续浮点数会按两位小数显示。这改变的是输出格式，不是把变量内部的数值永久舍入为两位。格式状态会继续影响之后的输出，直到再被修改；因此一个演示结束后，别把下一段输出的格式变化误认为计算错误。

使用 >> 读取 string 会在空白处停止。若名称含空格，例如 AIM Sentry，需要 getline。数字提取后流中可能残留换行，紧接着 getline 会读到空行；对不允许前导空白的名称，可使用 getline(cin >> ws, name)。若空行或前导空格本身有意义，则应有针对性地处理分隔换行。

完整课堂实验

L1_C03_D02.cpp · CPP-Classroom source

```
#include <iomanip>
#include <iostream>
#include <string>

int main() {
    double a = 0.0;
    char b = '\0';
    std::string c;
    std::cin >> a >> b >> c;
    std::cout << std::fixed << std::setprecision(2) << a << ' ' << b << ' ' << c
              << '\n';
    return 0;
}
```

标准输入：

```
3.14159 A mutsumi
```

执行推导

输入 3.14159 A mutsumi，输出 3.14 A mutsumi。c 只读一个无空白词；若输入尾部还有另一个词，它仍留在流中，尚未被这段程序消费。

常见错误及原因

setprecision(2) 单独使用时通常控制有效数字；与 fixed 联用才按这里的规则控制小数位数。

自检

输出两位小数之后，a 是否已经只剩两位小数？

参考解释： 没有。显示格式和存储数值是两件事，后续运算仍使用存储的 double 值。

本章配套例题

ID	题目
S03	读取两个整数
S04	读取带空格的名称
S05	数字之后读取整行
S06	输入失败检测

04 运算符的逐步计算

原稿定位: calculate：依照原文件内部顺序

4.1 余数、整数除法与转换时机

原 calculate 首先并排输出 3%2、3/2 和 3.14/2。% 取得整数除法的余数，3%2 为 1；两个 int 相除仍按整数除法计算，3/2 为 1；至少一个操作数为 double 时，3.14/2 得到近似的 1.57。类型先决定运算规则，之后才有输出。

如果要得到 3 除以 2 的实数商，必须在除法之前把至少一个操作数转换成 double。static_cast<double>(3/2) 先计算整数商 1，再把 1 转为 1.0，已经丢失的部分不会回来。表达式的求值顺序与类型，比最后接收结果的变量类型更重要。

整数除法向零截断，因此 -3/2 为 -1，不能把 Python 的 // 语义直接搬来。除数为 0 不在合法运算范围内，必须先验证。带符号整数溢出也不是自动扩容；给乘法增加宽类型操作数，例如 1LL*a*b，可以在需要时让中间运算使用 long long。

完整课堂实验

L1_C04_D01.cpp · CPP-Classroom source

```
#include <iomanip>
#include <iostream>

int main() {
    std::cout << 3 % 2 << ' ' << 3 / 2 << '\n';
    std::cout << std::fixed << std::setprecision(2) << static_cast<double>(3) / 2
              << ' ' << static_cast<double>(3 / 2) << '\n';
    return 0;
}
```

执行推导

第一行是 1 1。第二行第一项先转换得到 1.50，第二项先整除得到 1，再转换显示 1.00。

Step	State / operation	Result
`3 / 2`	`int / int`	`1`
`static_cast<double>(3) / 2`	`double / int`	`1.5`
`static_cast<double>(3 / 2)`	`int division, then conversion`	`1.0`

常见错误及原因

不能通过把结果变量改成 double 来修复已经在 int 中发生的除法或溢出。

自检

double result = 5 / 2 的 result 是多少?

参考解释： 2.0，因为右边先执行整数除法。可写 5/2.0 或先转换一个操作数得到 2.5。

4.2 赋值、比较与 bool 输出

原例先令 x=10、y=1，再观察相等、大小关系，随后执行 y=x。单个等号把右边的值赋给左边的对象，双等号才比较两边是否相等。赋值改变状态，比较产生 bool；两者都能出现在表达式里，但不能因为 “编译没报错” 就认为含义相同。

bool 只有 true 与 false 两种逻辑值。默认通过 cout 输出时通常显示 1 与 0；使用 boolalpha 可显示单词。x<=y、x!=y 等比较的结果都是 bool，而不是 Python 风格的另一种数值记录。给中间结果取一个有意义的名字，有助于把复杂判断分解成可以检查的部分。

不要把多个数学比较直接连写。0<=x<=100 在 C++ 中先得到 0<=x 的 bool，再把这个布尔结果与 100 比较，因此不能表达区间。正确形式是 0<=x && x<=100。原例中比较与赋值的先后关系也说明：输出必须结合那一时刻的变量值推导。

完整课堂实验

L1_C04_D02.cpp · CPP-Classroom source

```
#include <iostream>

int main() {
    int x = 10, y = 1;
    std::cout << std::boolalpha << (x == y) << '\n';
    y = x;
    std::cout << (x == y) << ' ' << (y != x) << '\n';
    return 0;
}
```

执行推导

第一次比较为 false。赋值使 y 从 1 变为 10，第二行变成 true false；x 没有因 y=x 而改变。

Step	State / operation	Result
initial	x=10	y=1
y=x	x=10	y=10
comparison	x==y: true	y!=x: false

常见错误及原因

if (x=0) 会把 x 改为 0，再把赋值结果当条件使用。开启编译警告能帮助发现这类错误，但仍要理解语义。

自检

执行 y=x 后，再把 x 改为 3，y 会自动变成 3 吗？

参考解释： 不会。普通整数赋值复制当时的值，不建立两变量间的持续绑定。引用将在第二课专门说明。

4.3 逻辑与、或、非和短路

原例用 x>3 && x<15 表达两个条件都成立，用 || 表达至少一个条件成立，用 ! 取逻辑反。先独立读懂每个比较，再组合它们。中文“与”“或”的描述必须与代码吻合，不能像旧演示中的某一条注释那样把 y>15 写成 y<15 后仍沿用原解释。

&& 和 || 从左到右短路。左边已确定整体结果时，右边不再求值。判断 p!=nullptr && *p>0 就依靠这个性质避免在空指针上解引用；第一课先用除数非零与商的条件理解同一规则，第二课再回到指针。

在复合条件里使用括号把规则明确写出来。! 与比较结合时尤其容易看错。对于机器人门控，先列出“血量大于零”“热量低于阈值”“确实有目标”三个独立条件，再写合取，往往比直接堆一长行表达式更容易核对边界。

完整课堂实验

L1_C04_D03.cpp · CPP-Classroom source

```
#include <iostream>

int main() {
    int a = 9, b = 0;
    bool valid = b != 0 && a / b > 2;
    std::cout << std::boolalpha << valid << '\n';
    std::cout << !(3 > 1 && 3 < 5) << '\n';
    return 0;
}
```

执行推导

b!=0 为 false，右边 a/b 不执行，所以第一行安全输出 false。第二行括号内为 true，取反后为 false。

常见错误及原因

把保护条件放到危险运算后面没有保护作用：a/b>2 && b!=0 已经先除以零。

自检

false && expensive() 会调用 expensive 吗？

参考解释： 不会。左操作数已经使整个 && 表达式为 false。

4.4 数学函数、自增与复合赋值

原 calculate 的后半部分依次演示 sqrt、pow、ceil、floor，再演示 x++、x+=5、x--。sqrt(2) 返回平方根的浮点近似，pow(3,3) 做幂运算，ceil 和 floor 分别沿数轴向上或向下取整。它们返回的通常仍是浮点值，名称中的 “取整” 不等于返回类型自动变为 int。

负数能帮助区分取整规则：ceil(-2.5) 为 -2，floor(-2.5) 为 -3，而整数转换通常截去小数部分得到 -2。不要凭 “去掉小数点” 理解 floor。浮点结果也不一定能被十进制文本精确表示，比较近似测量值时要使用问题定义的容差。

x+=5 的效果是用原 x 加上 5，再写回 x。单独成句的 ++x 与 x++ 都使 x 增加 1；区别在于表达式本身提供更新前还是更新后的值。入门阶段先把状态更新写成独立语句，避免在同一表达式里多次读写同一个变量制造难以理解的顺序问题。

完整课堂实验

L1_C04_D04.cpp · CPP-Classroom source

```
#include <cmath>
#include <iostream>

int main() {
    std::cout << std::ceil(-2.5) << ' ' << std::floor(-2.5) << '\n';
    int x = 10;
    x++;
    x += 5;
    x--;
    std::cout << x << '\n';
    return 0;
}
```

执行推导

第一行输出 -2 -3。x 的状态依次是 10、11、16、15，最终输出 15。应在每条语句后记录状态，而不是只看运算符个数猜结果。

Step	State / operation	Result
initial	x	10
x++	x	11
x+=5	x	16
x--	x	15

常见错误及原因

平方很小的整数时直接 x*x 清楚且可使用整数类型；不要以为 pow 返回整数就天然避开精度或范围问题。

自检

floor(-1.2) 和 static_cast<int>(-1.2) 一样吗？

参考解释： 不同，前者为 -2，后者为 -1。

本章配套例题

ID	题目
S09	整数除法与转换
S10	秒数拆分
S11	三次测量平均值
S12	大整数乘法
S13	摄氏转华氏
S14	欧氏距离
S15	浮点容差比较
S16	非负向上整除

05 条件与 for 循环

原稿定位: if_and_for

5.1 if、else if 与互斥分支

原 if_and_for 先让学员输入一个整数，再按正数、负数、零分成三个互斥结果。if 后的括号放条件，条件成立才执行对应大括号内的语句。else if 只会在前面的条件不成立时继续判断，最后的 else 接住剩余情况。

分支顺序表达了业务优先级。若血量为零必须先报告 DEAD，就应先处理这一特殊情况，再判断低血量。多个独立 if 不会自动互斥，可能在一轮执行多个动作。先写出区间或优先级表，再把每一行转成分支，可以避免交叠与遗漏。

即使分支只包含一条语句，也建议在讲义示例中保留大括号。这样后续追加输出或状态更新时，不会误以为新语句也受条件控制。调试时至少测一个正数、一个负数和零；测了两个正数并没有覆盖三个分支。

完整课堂实验

`L1_C05_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int x = 0;
    std::cin >> x;
    if (x > 0) {
        std::cout << "POSITIVE\n";
    } else if (x < 0) {
        std::cout << "NEGATIVE\n";
    } else {
        std::cout << "ZERO\n";
    }
    return 0;
}
```

标准输入：

```
0
```

执行推导

输入 0 时，x>0 与 x<0 都为 false，只有 else 中的 ZERO 被输出。

常见错误及原因

if (x>0); 后面的分号会形成空语句，后续大括号不再是这个 if 的受控语句。

自检

把 else if 改为独立 if 后，最后一个 else 与哪个 if 配对?

参考解释： 它与语法上最近且尚未配对的 if 对应，因此逻辑可能与原来的三选一完全不同。

5.2 for 的初始化、条件与更新

原例从 i=1 循环到 i=5，打印“第几次循环”。for 头部的三个部分分别负责起始状态、继续条件和每轮后的更新。执行顺序是初始化一次，检查条件，执行循环体，更新，再回到条件检查。条件第一次就为 false 时，循环体一次也不执行。

数次数与遍历下标采用不同边界。打印五次可以用 1..5，但长度为 5 的数组下标是 0..4，通常写 i<5。不能因为看到“五个元素”就写 i<=5。半开区间 [0,n) 的长度正好是 n，是后续数组循环最重要的约定。

循环中的累加器要在循环外初始化。每轮结束时，它保存已经处理部分的结果，这就是一个可用来检查正确性的循环不变量。n=0、n=1 与一般 n 能迅速揭示初始化或边界错误。扩展练习再引入 while、break 与 continue，主线先把原来的 for 循环彻底讲清。

完整课堂实验

L1_C05_D02.cpp · CPP-Classroom source

```
#include <iostream>

int main() {
    int n = 0;
    std::cin >> n;
    int sum = 0;
    for (int i = 1; i <= n; ++i) {
        sum += i;
        std::cout << i << ':' << sum << '\n';
    }
    return 0;
}
```

标准输入：

```
3
```

执行推导

输入 3，三轮分别输出 1:1、2:3、3:6；输入 0 无输出，因为第一次条件检查就失败。

Step	State / operation	Result
iteration 1	i=1	sum=1
iteration 2	i=2	sum=3
iteration 3	i=3	sum=6
next condition	i=4	4<=3: false

常见错误及原因

如果在循环体里再次定义 int sum = 0，每轮会建立新的零值累加器，历史总和就丢失了。

自检

长度为 n 的数组使用 i=0; i<=n; ++i 会访问几次？

参考解释： n+1 次，最后一次下标 n 越界。应使用 i<n。

本章配套例题

ID	题目
S17	血量三档
S18	闭区间判断
S19	开火门控
S20	短路保护除法
S21	switch 指令
S22	三值中位数
S23	闰年判断
S24	安全规则优先级
S25	循环累加
S26	哨兵值输入
S27	过滤与提前停止
S28	整数各位和
S29	因子枚举
S30	素数判断
S31	矩形输出
S32	连续命中计数

06 数组、std::array 与范围遍历

原稿定位: score；score_new；shuzu

6.1 固定数组与零初始化

原程序的 score[5]={94,87,50} 声明长度为 5 的整数数组，只显式提供前三个初值。剩余元素按聚合初始化规则变为 0，所以依次读取五项得到 94、87、50、0、0。这不是“所有局部数组都会自动清零”；如果没有初始化，局部基本类型数组的元素并不保证为零。

数组名加方括号选择一个元素，索引从 0 开始。score[0] 是第一项，score[4] 是最后一项，score[5] 已不属于这个数组。原生数组的下标操作不会替你检查边界，越界可能表现为错误值、崩溃或暂时看起来正常，最后一种情况也不代表程序正确。

旧注释把大型数组放到全局作为普遍建议，这里需要修正。自动对象占用的存储与实现、平台、线程栈设置有关，不能给所有学生一个固定栈容量结论。很大的、运行时才知道长度的数据优先使用 std::vector；不要通过扩大共享全局状态来回避所有存储设计问题。

完整课堂实验

L1_C06_D01.cpp · CPP-Classroom source

```
#include <iostream>

int main() {
    int score[5] = {94, 87, 50};
    for (int i = 0; i < 5; ++i) {
        std::cout << i << ':' << score[i] << '\n';
    }
    return 0;
}
```

执行推导

第三个显式值位于索引 2。索引 3 与 4 输出 0，循环总共执行五次，绝不会访问索引 5。

Step	State / operation	Result
score[0]	explicit	94
score[1]	explicit	87
score[2]	explicit	50
score[3]	value-initialized	0
score[4]	value-initialized	0

常见错误及原因

“长度是 5”和“最后下标是 5”不是同一件事。

自检

如果只写 int score[5]; 放在 main 中，还能断言最后两项为 0 吗？

参考解释： 不能，没有初始化时不具备这个保证。

6.2 原生数组复制与 std::array 赋值

原 shuzu 用一个循环把 new_scores 的每项复制给 score，因为原生数组不能用整体赋值运算符直接复制。数组长度属于声明，创建后不会因为赋值而改变。想更新全部元素，就按合法下标逐项写入，或者使用适合整体值语义的容器。

`std::array<int,5>` 把固定长度数组包装成标准容器。它支持 size、at、范围遍历和同类型整体赋值。`array<int,5>` 与 `array<int,4>` 是不同类型，因此“长度相同”并不是运行时再随便补上的条件。原例中的 score_new={1,2,3,4,5} 是给 std::array 赋值，不能据此说原生数组也能同样写。

at 会在越界时抛异常，[] 不做对应保证。课堂中先保证下标合法，再观察两种接口的区别。不要把异常当作每次循环都需要依赖的正常退出方式，先写对 i<size 的边界。

完整课堂实验

`L1_C06_D02.cpp` · CPP-Classroom source

```
#include <array>
#include <iostream>

int main() {
    std::array<int, 5> scores = {94, 87, 50};
    std::array<int, 5> replacement = {80, 85, 90, 95, 100};
    scores = replacement;
    std::cout << scores.size() << ' ' << scores.at(0) << '\n';
    return 0;
}
```

执行推导

整体赋值后 scores 的五个元素都来自 replacement，输出 5 80。两个容器仍是独立对象，随后修改 replacement 不会自动改变 scores。

常见错误及原因

不要把 std::array 的整体赋值规则推广到原生数组。

自检

`array<int,5>` 的 size 是否可能在 push_back 后变成 6？

参考解释： 不能，它没有用于增长的 push_back 接口；需要动态长度时选择 vector。

6.3 范围 for 中的副本与修改

原 shuzu 的 for(int j:score_new) 每轮把一个元素的值复制到 j，再执行循环体。它适合只读地输出小型整数。把 j 改成 0 不会把容器中的元素改为 0，因为 j 是本轮局部副本。先认识这个复制行为，第二课再正式解释引用符号。

如果确实需要修改原元素，可以写 int& j。只读遍历较大对象通常写 const std::string&，既避免不必要的复制，也表达不修改元素。原课程第二课正是用 Band& 遍历 vector<Band>，两处写法应放在同一个“副本还是原对象”的问题下理解。

vector 可以在运行时追加元素，适合读取未知数量的数据。增长可能引起存储重新分配，使指向原元素的引用、指针或迭代器失效。这是后续工程补充知识，不应在第一遍数组输出前抢走主线，但在题库用到动态容器时必须说明。

完整课堂实验

L1_C06_D03.cpp · CPP-Classroom source

```
#include <array>
#include <iostream>

int main() {
    std::array<int, 3> a = {1, 2, 3};
    for (int value : a) {
        value = 0;
    }
    for (int &value : a) {
        value *= 2;
    }
    for (int value : a) {
        std::cout << value << '\n';
    }
    return 0;
}
```

执行推导

第一个循环只修改副本，a 仍为 1、2、3；第二个循环修改元素，a 变为 2、4、6。最后输出这三个值。

常见错误及原因

不要在保存了 vector 元素引用后随意继续 push_back，再假定旧引用仍然有效。

自检

把第二个循环中的 & 删除，最后输出会是什么？

参考解释： 1、2、3，因为两个更新循环都只修改副本。

本章配套例题

ID	题目
S33	array 初始化
S34	数组最大值
S35	vector 动态追加
S36	索引合法性
S37	复制与修改
S38	相邻差分
S39	稳定过滤
S40	循环右移

07 字符串的查、改、增、删

原稿定位: str_modern：保留 mygo 与成员名语境

7.1 字符串与安全索引

原课程用 mygo="soyo,anon,taki,rana,tomori" 把标准字符串操作串起来。std::string 是一个管理字符序列的对象，可以改变长度，不需要先手工分配一个足够大的 C 字符数组。字符串中的逗号目前只是普通字符，还不会自动形成五个成员对象。

at(3) 访问从 0 计数的第四个字符，所以初始文本里得到 o。字符串是否为空，要先用 empty 检查，再使用 front 或 back。size 返回字符存储单元数量；当内容采用 UTF-8 编码时，它统计的字节数不能直接当作汉字数量。

同一个变量在不同操作之后保存不同文本。讲义会在每一步列出当前值，避免原稿中“代码已换了字符串，输出注释还停在另一份演示”的问题。练习时也应每做一次 replace、insert 或 erase 就观察一次结果。

完整课堂实验

L1_C07_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string mygo = "soyo,anon,taki,rana,tomori";
    std::cout << mygo.at(3) << '\n';
    std::cout << mygo.size() << '\n';
    return 0;
}
```

执行推导

初始文本共有 26 个 ASCII 字符，at(3) 输出 o。逗号计入长度，最后没有隐含的成员分割。

常见错误及原因

字符下标与“第几个成员”不同，at(3) 不会得到第四位乐队成员。

自检

为什么中文字符串的 size 可能大于肉眼看到的汉字数?

参考解释： UTF-8 的一个汉字通常占多个 char 字节，size 统计存储单元而非用户感知字符。

7.2 find 与 npos

find("anon") 返回第一次匹配的起始位置。原始字符串中 soyo 有四个字符，后面一个逗号，因此 anon 从索引 5 开始。原稿“输出 10”的注释与这个文本不符，修订版以实际字符序列逐项推导。

未找到时返回 std::string::npos，这是一个特殊的 size_type 值。不能把它当合法下标，也不应先强制窄化到 int 再猜是不是 -1。使用 std::string::size_type 接收，再与 npos 比较，只有确实命中才继续 substr、replace 或 erase。

find 是字符串搜索，不是基于逗号字段的精确成员查询。搜索 ana 也可能命中更长名字中的一段。如果问题要求按完整字段匹配，应先定义分隔规则并拆分，不能把部分子串命中当作完整成员相同。

完整课堂实验

L1_C07_D02.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string mygo = "soyo,anon,taki,rana,tomori";
    std::string::size_type pos = mygo.find("anon");
    if (pos != std::string::npos) {
        std::cout << pos << '\n';
    }
    std::cout << std::boolalpha << (mygo.find("saki") == std::string::npos) << '\n';
    return 0;
}
```

执行推导

第一行输出 5，第二行输出 true，因为初始文本不含 saki。搜索本身不修改 mygo。

Step	State / operation	Result
soyo	indexes	0..3
,	index	4
anon	indexes	5..8
find("anon")	position	5
find("saki")	not found	npos

常见错误及原因

find 返回位置 0 也是成功。不能用 if(pos) 区分成功与失败。

自检

在 "anon,soyo" 中 find("anon") 返回 0，它是否表示没找到？

参考解释： 不是，0 表示匹配从第一个字符开始，只有 npos 表示未找到。

7.3 replace 与长度变化

原 str_modern 把 anon 替换成 saki。replace(pos,count,text) 的 count 是要删掉多少个原字符，而不是结束下标。起点为 5、删除长度为 4，就恰好移走 anon 的四个字符；新的文本长短可以不同，字符串会相应增长或缩短。

旧演示直接写 replace(find("anon"),4,"saki")。若文本已没有 anon，find 返回 npos，后续 replace 不能把它作为正常起点。讲义版本保留操作意图，但加入命中判断；这是修复错误边界，不改变原课程的“先查再改”逻辑。

替换可能改变所有后续字符的位置。若先保存了两个位置，再修改前面的文本，后面的旧位置可能不再指向原来的内容。对初学者，最稳妥的过程是每次修改后重新根据当前文本查找。

完整课堂实验

L1_C07_D03.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string mygo = "soyo,anon,taki,rana,tomori";
    std::string::size_type pos = mygo.find("anon");
    if (pos != std::string::npos) {
        mygo.replace(pos, 4, "saki");
    }
    std::cout << mygo << '\n';
    return 0;
}
```

执行推导

输出 soyo,saki,taki,rana,tomori。这次新旧名字同为四个字符，所以总长度不变；改成另一个长度时不能继续作这个假设。

常见错误及原因

count=4 表示四个字符，不是从 pos 处理到绝对索引 4。

自检

把 anon 换成 A 后，原来 taki 的起点会提前多少？

参考解释： 提前三个字符，因为删除四个再插入一个，长度减少三。

7.4 拼接、insert、erase 与 clear

原稿接着用 += 在末尾追加文本，再用 insert(0,...) 在开头插入。+= 适合追加到尾部，insert 让我们明确指定插入位置。逗号分隔名单的标点也要由代码负责，字符串不会自动给新成员补分隔符。

删除 saki 时，若只擦去名字的四个字符，会留下相邻逗号。原例删除五个字符，意图连同后面的逗号一起移除。这个长度依赖“saki 后面确有逗号”的当前文本，不能无条件推广到最后一个字段。先确认边界再决

定删除范围。

clear 使字符串内容变为空，随后 size 为 0；它并不销毁变量本身，也不承诺立刻向操作系统归还所有已分配存储。把“值为空”和“对象生命周期结束”分开，第二课讨论对象时会继续使用这个区别。

完整课堂实验

L1_C07_D04.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string text = "soyo,saki,taki";
    text += ",rana";
    text.insert(0, "anon,");
    std::string::size_type pos = text.find("saki,");
    if (pos != std::string::npos) {
        text.erase(pos, 5);
    }
    std::cout << text << '\n';
    text.clear();
    std::cout << text.size() << '\n';
    return 0;
}
```

执行推导

依次追加 rana、前插 anon、删除 saki 及其后逗号，得到 anon,soyo,taki,rana。clear 后输出 0。

常见错误及原因

查不到目标时不执行 erase；字段处于开头、中间、末尾时，应分别分析分隔符归属。

自检

clear 之后还可以再次执行 text += "anon" 吗？

参考解释： 可以，text 对象仍然存在，只是当前字符串为空。

本章配套例题

ID	题目
S41	字符串长度与字符
S42	查找子串
S43	安全替换首次命中
S44	逗号字段拆分
S45	ASCII 小写转大写
S46	单词计数
S47	严格整数解析
S48	字符串回文

08 第一课整合与结构体预告

原稿定位: 原 README 的 struct 预告；不提前替代第二课函数主线

8.1 从平行变量到一条记录

原 README 在第一课列出了结构体，但原示例未把它单独展开。这里补上最小过渡：当一条机器人记录同时包含编号、血量与名称时，把它们放进一个 struct，比依靠三组互相约定下标的数组更容易读懂。struct 在第二课也会成为进入 class 的直接起点。

点运算符选择一个对象的数据成员。Frame a 与 Frame b 是两个对象，修改 a.hp 不会自动修改 b.hp。这个“每个对象有自己的状态”会在 Band 的不同乐队实例上再次出现。当前先使用公开字段，不在第一课提前插入完整的访问控制课程。

第一课的收束任务是读入、计算、判断、遍历、输出。进阶题库保留更多解析与仿真任务，但算法优化、函数重载、虚函数或资源所有权都不能反过来挤占原课程顺序。第二课会先回到 test_add，再进入引用、指针、交换实验与类。

完整课堂实验

L1_C08_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

struct Frame {
    int id = 0;
    int hp = 0;
    std::string name;
};

int main() {
    Frame a = {7, 80, "Sentry"};
    Frame b = a;
    a.hp = 30;
    std::cout << a.id << ':' << a.hp << '\n';
    std::cout << b.id << ':' << b.hp << '\n';
    return 0;
}
```

执行推导

复制时 b 得到当时的记录值；随后只修改 a，输出 7:30 与 7:80。

常见错误及原因

结构体末尾的分号属于类型定义，不能因为后面有大括号就省略。

自检

如果保存五台机器人，应写五组独立变量，还是 vector<Frame>?

参考解释： 记录集合适合用容器保存 Frame，再按对象遍历；固定数量也可使用 array<Frame,5>。

本章配套例题

ID	题目
S65	结构体遥测帧
S66	最近目标
S67	枚举方向
S68	二维数组行和
S69	二维坐标更新
S70	固定窗口均值
S71	时间戳去重
S72	排序和平局
S73	有效数据统计
S74	运行长度编码
S75	受限命令解释器
S76	热量仿真
S77	网格碰撞
S78	丢帧告警
S79	Debug 循环边界
S80	任务汇总报表